Vision transformers and multimodal retrieval

COCO

LECTURE 23 | GÉRON CH. 15–16

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Part V built retrieval over text and over user behaviour. Both reduced to the same object: **two encoders and an inner product**.

- Lecture 20 — a query encoder and a document encoder, retrieved by inner product
- Lecture 22 — a user tower and an item tower, the same architecture with users in place of queries

> **TODAY THE TWO TOWERS TAKE DIFFERENT MODALITIES**
>
> One encodes **images**, the other **text**, and a photograph and its caption must land close together. Everything about the retrieval is unchanged; what is new is that the two encoders never see the same kind of input.

Today

1. The vision transformer: an image as a sequence of patches (15 min)
2. Two encoders trained separately, and the discovery that their spaces are unrelated (15 min)
3. **The mathematics** — the contrastive objective and its temperature (25 min)
4. A jointly trained dual encoder: zero-shot classification and text-to-image retrieval (25 min)

PART ONE

The brief

15 minutes

The stakeholder

A retailer’s catalogue team. Their catalogue has one property that breaks every search system they have bought:

> **THE ENTRIES ARE PHOTOGRAPHS**
>
> Product photos arrive from suppliers with an identifier, a price and a picture. Written descriptions are inconsistent, often absent, and never written by the person searching.

Customers type sentences. The catalogue stores pixels. Nothing in the system converts one into the other.

What they are asking for

Input	Output
A sentence typed by a customer	A ranked list of catalogue entries
“a red bus on a city street”	the entries whose *picture* shows that

- No labels. Nobody is going to annotate the catalogue
- No fixed vocabulary. The query is whatever the customer typed
- The answer must be a **ranking**, not a yes/no

This is the first brief in the course whose input and output are not the same kind of object.

Why the obvious thing does not apply

Keyword search needs text on both sides. So does the semantic search you built in the previous application: it embeds a sentence and compares it to other *sentences*.

> **THERE IS NO TEXT ON THE CATALOGUE SIDE**
>
> You can index descriptions when they exist. But an entry with no description is invisible to a text index, no matter how good the index is — not badly ranked, absent.

Hold that thought. The next lecture counts how many entries it costs us, and repairs it.

Where this sits in the book

Idea	Chapter
Transformers, attention, the encoder stack	15
Vision transformers: an image as a sequence of patches	16
Text–image models and zero-shot prediction	16
Sentence embeddings and semantic search	15

Chapter 16 is the last chapter in scope. This is the last application, and it uses both halves of Part IV at once.

The data, and what we are ✗ not doing

The catalogue is drawn from **COCO** — the same corpus as the detection application, and one the book names.

> **WE ARE NOT DOWNLOADING COCO**
>
> The full release is about 20 GB. Nothing in this lecture needs it, and a lecture that cannot be re-run on a laptop is a lecture nobody re-runs.

What we take instead: the split index — a small CSV listing images and their human-written captions — and then the images themselves, *one at a time*, only the ones we use.

The corpus, and its size

200

catalogue entries
32.6 MB on disk

- COCO 2014 validation images, taken in order of image id, so everyone's catalogue is the same catalogue
- 5 independent human captions per image — written by people who were not shown each other's
- Every retrieval number today is measured against **200 candidates**

✗ Say the candidate-set size every time you say a recall. A recall without it is not a measurement.

Sixteen of the two hundred entries

16 of the 200 catalogue entries — COCO validation images, each with five human captions we hold back

Ordinary photographs of ordinary things — and five sentences each, written by five different people, which the search system never sees.

What one entry looks like

```
entry = {
    "sku":      "CAT-000164",
    "file":     "COCO_val2014_000000000164.jpg",
    "captions": [
        "A black and white photograph of a man on a bench.",
        "A man sitting alone on a park bench reading.",
        "An older gentleman reads a newspaper outdoors.",
        # ... five in total, five different people
    ],
}
```

The captions are *evidence*, not part of the product. They let us score a system that never sees them.

Two different uses for five captions

Caption	Role	Who sees it
#1	the entry's *description*, when it has one	the text index
#2	the customer's *query*	the search box
#3–5	held back	nobody, today

Using two different people's sentences about the same picture makes this a paraphrase problem rather than a lookup. If we used the same sentence on both sides, a hash table would score 100%.

A second corpus, for a different job

Step 4 of the working loop: **test against a case whose answer you know**. The catalogue has no labels, so it cannot play that role.

Corpus	Size	Job
COCO catalogue	200 images	the real task: rank entries for a query
CIFAR-10 test	2,000 images	the known-answer check: ten labels we have

CIFAR-10 is 32×32. That is a hard setting for any model trained on web photographs, and we will see the cost.

Two things you can already build

Encode an image

A vision transformer cuts the image into patches, embeds each, and runs a transformer encoder over the sequence. The `[CLS]` vector is the image.

Chapter 16. You met the convolutional route in Lecture 12.

Encode a sentence

A transformer encoder over subword tokens, mean-pooled over positions. The vector is the sentence.

Chapter 15. You built exactly this in the previous application.

So: encode the images, encode the query, take the cosine. What could go wrong?

An image as a sequence

A transformer takes a sequence. An image is a grid. So cut it up:

1. Split the image into fixed patches — 16×16 pixels is the usual choice
2. Flatten each patch and project it linearly to d_{model}. That projection *is* the embedding table of Lecture 17, with patches in place of tokens
3. Add a positional encoding, because Lecture 18 established that attention has no notion of order — and a grid has two
4. Run the standard transformer blocks

A 224×224 image at 16×16 patches is $14 \times 14 = 196$ tokens. Well within reach of the quadratic cost, which is exactly why this patch size and not a smaller one.

What a ViT gives up

THE INDUCTIVE BIAS IS GONE

Lecture 12 derived what convolution assumes: locality, and translation equivariance from weight sharing. A vision transformer assumes **neither**. Every patch can attend to every other from the first layer, and nothing tells it that adjacent patches are related.

- On small corpora a convolutional network wins, because its assumptions are true and free
- On large ones the transformer wins, because it can *learn* whichever relations actually hold rather than being told
- The crossover is at a corpus size most people do not have — which is why pretrained ViTs are used far more often than they are trained

The same trade as everywhere: assumptions substitute for data, and stop paying when data is plentiful.

PART TWO

A metric, and a baseline to anchor it

15 minutes

What would “it works” even mean?

The system returns a ranked list. So the quantity that matters is **where the right entry landed**.

$$r_q \;=\; 1 + \#\{\, j \neq q^\star : s(q, j) \geq s(q, q^\star) \,\}$$

THE RANK OF THE RELEVANT ENTRY FOR QUERY Q

Ties count *against* us: a tie is scored as the worse rank. A metric that flatters a degenerate score matrix is not a metric.

Recall@k

$$\mathrm{R@}k \;=\; \frac{1}{Q}\sum_{q=1}^{Q}\mathbf{1}\left[\,r_q \le k\,\right]$$

THE SHARE OF QUERIES WHOSE ANSWER IS IN THE TOP K

With exactly one relevant entry per query, recall and precision at *k* carry the same information, and recall is the one the literature reports. We will read R@1, R@5 and R@10.

R@1 is the number the product actually cares about: it is the top of the page.

Why not accuracy

- Accuracy needs a decision. There is no decision here — there is an ordering
- A threshold on similarity would import all of Lecture 3's problems and add a new one: the scale of a cosine is not comparable between models
- Recall@k is *rank-based*, so it is invariant to any monotone transformation of the score. That is exactly the invariance we want

> **ONE NUMBER IS NOT ENOUGH**
>
> R@1 alone hides how badly the failures fail. We also record the median rank and the mean reciprocal rank, so a system that puts the answer at 2 is distinguishable from one that puts it at 180.

Mean reciprocal rank

$$\mathrm{MRR} = \frac{1}{Q}\sum_{q=1}^{Q}\frac{1}{r_q}$$

Rank 1 contributes 1, rank 2 contributes 0.5, rank 100 contributes 0.01. It rewards being nearly right and stops rewarding it quickly — which matches a page of ten results.

The trivial baseline, computed rather than run

Rule 2 from the first lecture: **every number needs a baseline**. Here the baseline needs no experiment at all.

Shuffle the catalogue. One relevant entry among n, placed uniformly at random:

$$\Pr[r \le k] = \frac{k}{n}, \qquad \mathbb{E}[r] = \frac{n+1}{2}, \qquad \mathbb{E}\left[\frac{1}{r}\right] = \frac{H_n}{n}$$

RANDOM RANKING, EXACTLY

H_n is the harmonic number. No simulation, no seed, no error bar — this one is arithmetic.

What random ranking scores on our catalogue

Quantity	Value at n = 200
Recall@1	0.5%
Recall@5	2.5%
Recall@10	5.0%
Expected rank	100.5
MRR	0.029

✗ These scale with the catalogue. On 5,000 candidates R@1 would be 0.02%. Quote the size or quote nothing.

The plan

1. Encode the 200 images with a vision transformer
2. Encode the 200 queries with a sentence encoder
3. Cosine similarity, rank, measure Recall@k
4. Compare against 0.5%

Four steps, each of which you have written before. The interesting part is step 3, and it is interesting for a reason none of us has met yet in this course.

Step 1 — encode the images

```python
# an image-only encoder: ImageNet labels, no text anywhere in training
import torch
from transformers import AutoImageProcessor, ViTModel

proc = AutoImageProcessor.from_pretrained("google/vit-base-patch16-224")
vit = ViTModel.from_pretrained("google/vit-base-patch16-224",
                               add_pooling_layer=False).to(device).eval()

feats = []
with torch.no_grad():
    for i in range(0, len(images), 32):
        batch = proc(images=images[i:i + 32], return_tensors="pt").to(device)
        feats.append(vit(**batch).last_hidden_state[:, 0].cpu())  # [CLS]

V = torch.cat(feats).numpy()
assert V.shape == (200, 768), V.shape
```

`add_pooling_layer=False` because that checkpoint has no pooler weights — ask for one and you get a randomly initialised layer, silently.

Step 2 — encode the queries

```python
from transformers import AutoModel, AutoTokenizer

tok = AutoTokenizer.from_pretrained("sentence-transformers/all-MiniLM-L6-v2")
enc = AutoModel.from_pretrained("sentence-transformers/all-MiniLM-L6-v2") \
                .to(device).eval()

with torch.no_grad():
    b = tok(queries, return_tensors="pt", padding=True,
            truncation=True, max_length=128).to(device)
    h = enc(**b).last_hidden_state              # (200, tokens, 384)
    m = b["attention_mask"].unsqueeze(-1).float()
    T = ((h * m).sum(1) / m.sum(1)).cpu().numpy()   # mean pooling

assert T.shape == (200, 384), T.shape
```

Mean pooling over real tokens only — the mask matters, and forgetting it averages in the padding.

Step 3 — take the cosine

```
V.shape          # (200, 768)
T.shape          # (200, 384)

V @ T.T          # ValueError: matmul: dimensions 768 and 384 do not match
```

IT DOES NOT EVEN TYPECHECK

Reviewer question 3: *what is the shape here?* The image vectors live in 768 dimensions and the sentence vectors in 384. There is no cosine between them, because there is no inner product between them.

Fine — make the dimensions agree

Three ways, so that nobody can blame the fix:

#	Method	Justification
1	Random Gaussian projection, 768 → 384	Lecture 8: Johnson–Lindenstrauss
2	Keep the first 384 coordinates	crude, but it changes nothing that matters
3	Zero-pad the text vectors to 768	the other direction, for symmetry

Lecture 8 told us a random projection preserves the *image* geometry to within a stated tolerance — and it did: the pairwise cosines before and after the projection correlate at 0.86. It said nothing at all about aligning that geometry with anything else.

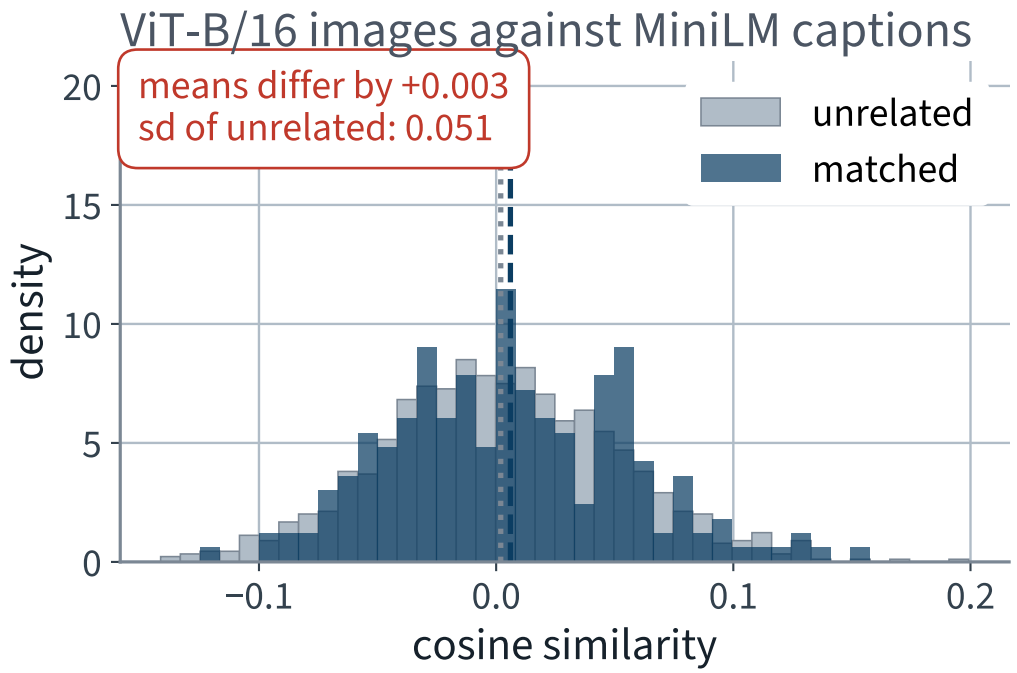
Now measure it — against what?

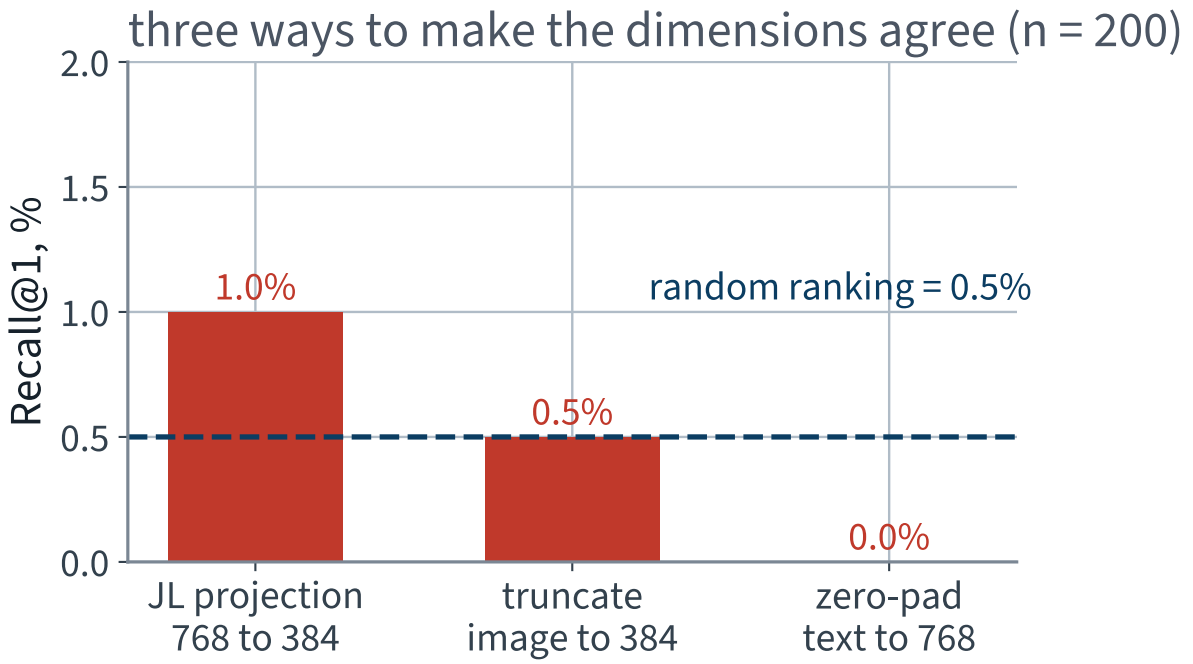
Reviewer discipline: a similarity is meaningless alone. Compute two numbers, always:

- the cosine between an image and **its own caption**
- the cosine between an image and **somebody else's caption** — 200 × 200 minus the diagonal, so nearly forty thousand of them

> **THE COMPARISON IS THE MEASUREMENT**
>
> If the two distributions sit on top of one another, the similarity carries no information about whether the pair matches.

The two spaces, measured





The matched pairs and the unrelated pairs are the same distribution, and all three fixes land on the random-ranking line.

The numbers behind that picture

Pairs	Mean cosine	sd
Image and its own caption	+0.0060	0.0499
Image and an unrelated caption	+0.0034	0.0514
✗ Difference	**✗ +0.0026**	Cohen's *d* = +0.052

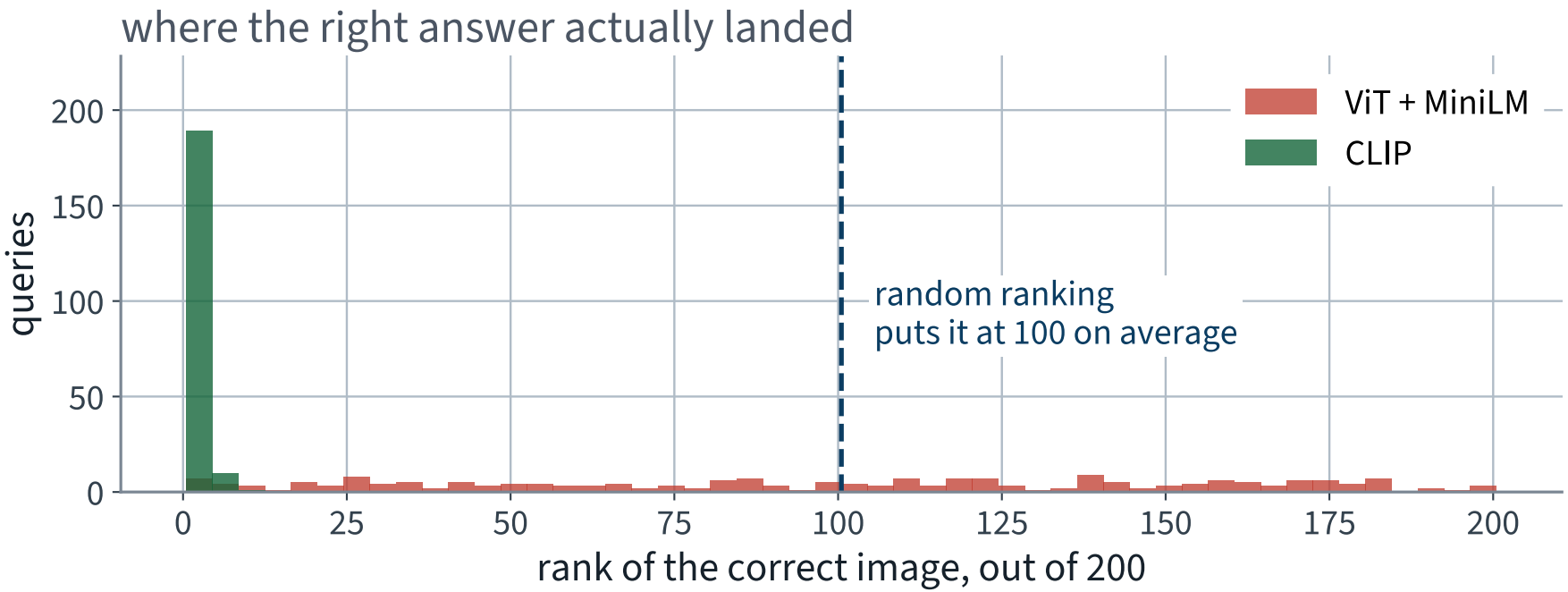
The gap between the two means is a small fraction of the spread of either. On the strength of the similarity alone, you cannot tell a matching pair from a random one.

And as a retrieval system

System	R@1	R@5	R@10	Median rank
Random ranking (exact)	0.5%	2.5%	5.0%	100
ViT + MiniLM, JL projection	1.0%	4.0%	6.0%	104
ViT + MiniLM, truncated	0.5%	2.0%	5.5%	80
ViT + MiniLM, zero-padded	0.0%	2.5%	5.5%	80

✗ Two of the 200 queries put the right image first. Shuffling is expected to get one. Two strong encoders, over a hundred million parameters between them, and that is the whole difference.

Where the right answer actually landed



A flat histogram is what “no information” looks like. Every rank is as likely as every other.

Say the result out loud

A picture of a bus and the sentence *“a red bus on a city street”* are **no closer** than that picture and a sentence about a sandwich.

BOTH ENCODERS WORK PERFECTLY

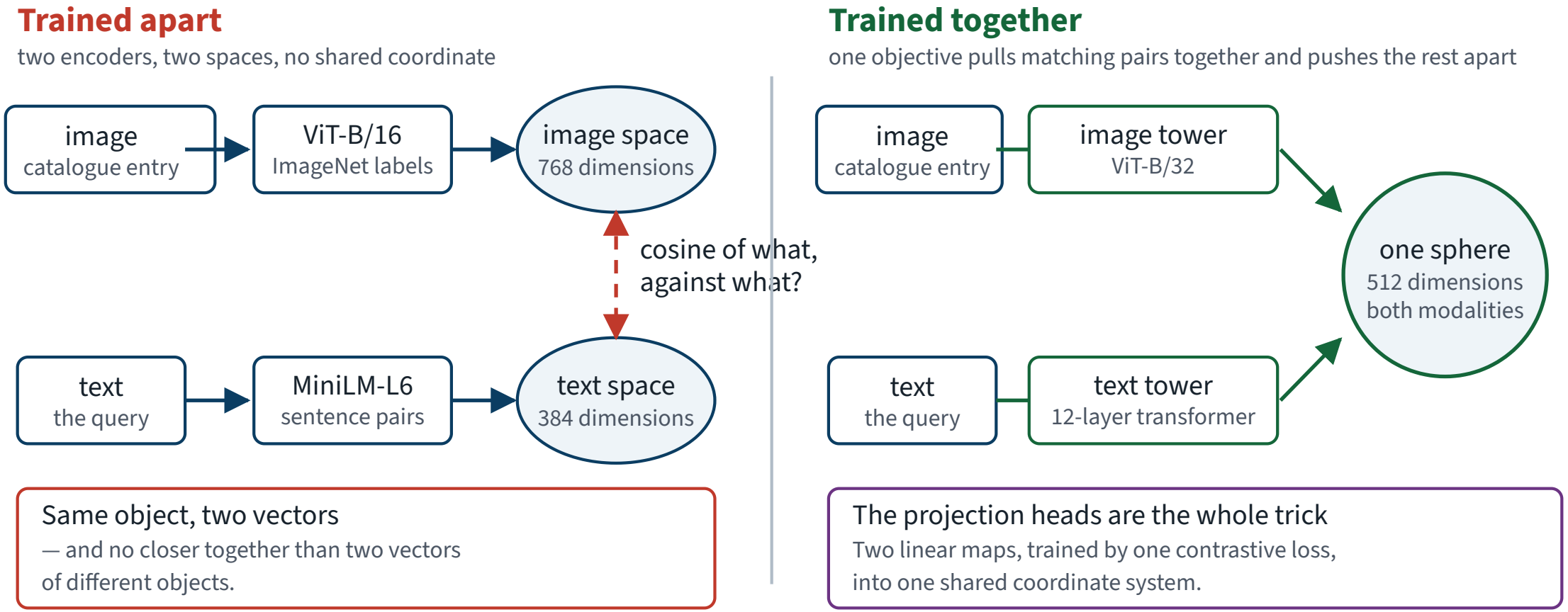
The ViT separates images from one another. MiniLM separates sentences from one another. Neither was ever shown a single (image, sentence) pair, so neither has any reason to place them anywhere in particular relative to each other.

Why, precisely

- An embedding space is defined *only* up to whatever the training objective constrained
- ImageNet classification constrains images relative to images
- Sentence-similarity training constrains sentences relative to sentences
- Nothing in either loss ever compared a vector from one to a vector from the other, so the relative orientation of the two spaces is **arbitrary**

> **A ROTATION YOU CANNOT SEE**
>
> Apply any orthogonal map to the image space: every image–image cosine is unchanged, so the ViT's loss is unchanged. The image–text cosines all change. The loss cannot distinguish those worlds, so it did not pick one.

Trained apart, trained together



The fix is not a better image encoder. It is a training signal that ever compared the two.

THE MATHEMATICS

The contrastive objective and its temperature

25 minutes · examinable

The question

Two encoders trained apart gave us 1.0%. Two encoders trained together gave us 73.0%.

What is the loss that did that?

> **FOUR DECISIONS, NONE OF THEM OBVIOUS**
>
> Where the vectors live; what an unrelated pair should score; how a cosine becomes a logit; and how many wrong answers the loss compares against. Each one is measurable, and we measure all four.

The setup

A batch of B pairs. Image tower f, text tower g, each ending in a linear map into $\mathbb{R}^d$:

$$\mathbf{u}_i = \frac{f(x_i)}{\|f(x_i)\|}, \qquad \mathbf{v}_j = \frac{g(t_j)}{\|g(t_j)\|}, \qquad S_{ij} = \mathbf{u}_i^\top \mathbf{v}_j$$

A B × B MATRIX OF COSINES

S_{ii} is a pair that belongs together. Every S_{ij} with $i \neq j$ is a pair that does not. The loss only ever sees this matrix.

1 · Why normalise at all

The unnormalised inner product factorises:

$$\langle a, b \rangle = \|a\| \, \|b\| \cos\theta$$

- Two quantities are mixed: *which direction* the encoder chose, and *how loudly* it said it
- Only the direction carries the semantics. The length is whatever the last linear layer happened to scale to
- Measured on our catalogue: the longest image embedding is 1.32× the shortest

Divide both by their norms and S_{ij} is a pure cosine, in $[-1, 1]$, comparable across pairs, models and runs.

What normalising costs, and buys

Costs

- One degree of freedom per vector: d numbers describe a point on S^{d-1}, which has dimension $d - 1$
- The model can no longer express confidence by shouting

Buys

- A bounded score, so a single temperature is meaningful for every pair
- A geometry we can reason about — and the reasoning is the next section

Confidence comes back through the temperature instead, as one number shared by the whole model rather than one per example.

2 · What should an *unrelated* pair score?

Matching pairs should score 1. Unrelated pairs should score … ?

THE ANSWER EVERYBODY GIVES FIRST

−1. Opposite meaning, opposite vector. It is the wrong answer, and the reason is geometry you already measured — in Lecture 8.

Why −1 is impossible to ask for

- For a given $\mathbf{u}$, there is **exactly one** unit vector at cosine −1: the point $-\mathbf{u}$
- A catalogue of 200 entries would need all 200 of its non-matching captions to sit on that one point — and then they would all be identical to each other
- Wanting every unrelated pair at −1 is asking for a configuration that does not exist for more than two items

> **THE RIGHT QUESTION**
>
> Not “what is the most different?” but “what does a pair with *no relationship at all* look like?” That is a question about a random vector, and Lecture 8 answered it.

Lecture 8, returning

In Lecture 8 the Johnson–Lindenstrauss bound worked because a random projection nearly preserves inner products. Underneath that is one fact about high dimensions:

> **CONCENTRATION ON THE SPHERE**
>
> Draw $\mathbf{u}, \mathbf{v}$ uniformly on S^{d-1}. Then $\mathbb{E}[\mathbf{u}^\top \mathbf{v}] = 0$ and $\mathrm{Var}(\mathbf{u}^\top \mathbf{v}) = 1/d$. Almost all of the sphere's area sits within $O(1/\sqrt{d})$ of the equator of any fixed point.

In high dimensions, two unrelated things are orthogonal, not opposite.

Where the $1/d$ comes from

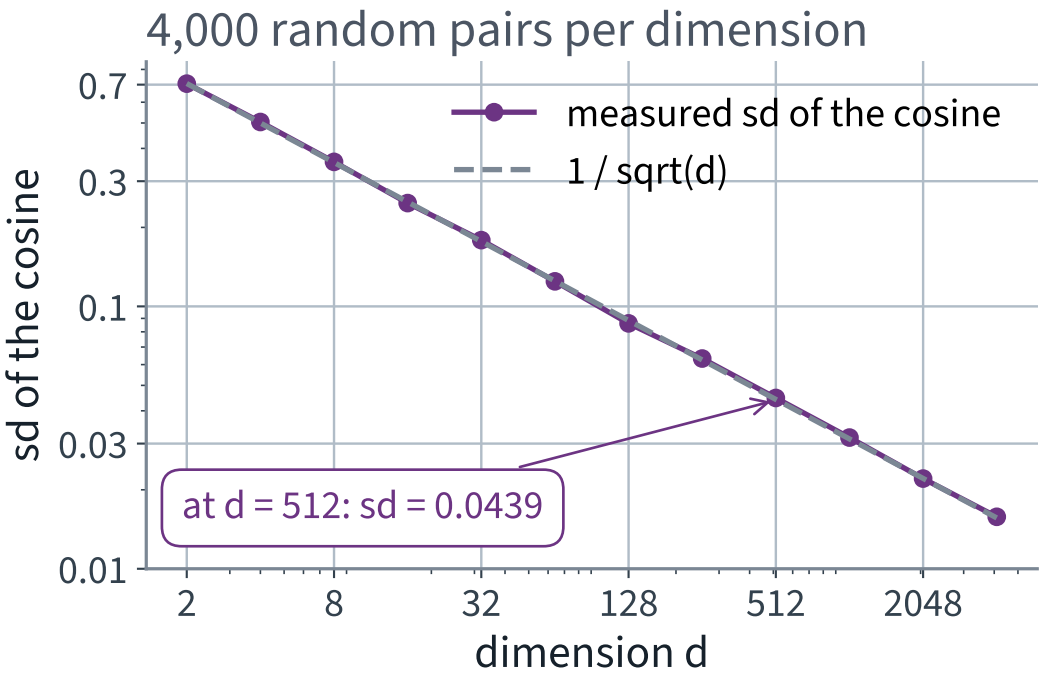
Fix $\mathbf{u} = e_1$ by rotational symmetry, and write $\mathbf{v} = \mathbf{z}/\|\mathbf{z}\|$ with $\mathbf{z} \sim \mathcal{N}(0, I_d)$. Then $\mathbf{u}^\top \mathbf{v} = z_1/\|\mathbf{z}\|$, and

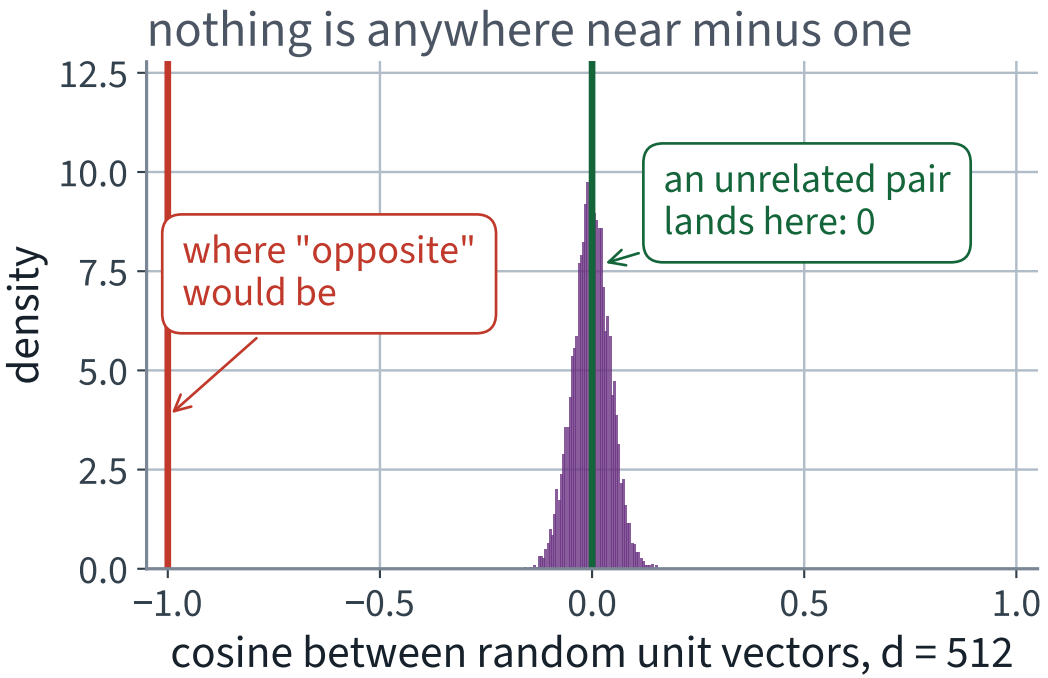
$$\mathbb{E}\left[\left(\frac{z_1}{\|\mathbf{z}\|}\right)^2\right] = \mathbb{E}\left[\frac{z_1^2}{\sum_k z_k^2}\right] = \frac{1}{d}$$

BY SYMMETRY: THE D COORDINATES SHARE THE TOTAL EQUALLY

The expectation is zero by the symmetry $\mathbf{v} \mapsto -\mathbf{v}$, so the standard deviation is exactly $1/\sqrt{d}$. No approximation anywhere.

Measured, over twelve dimensions





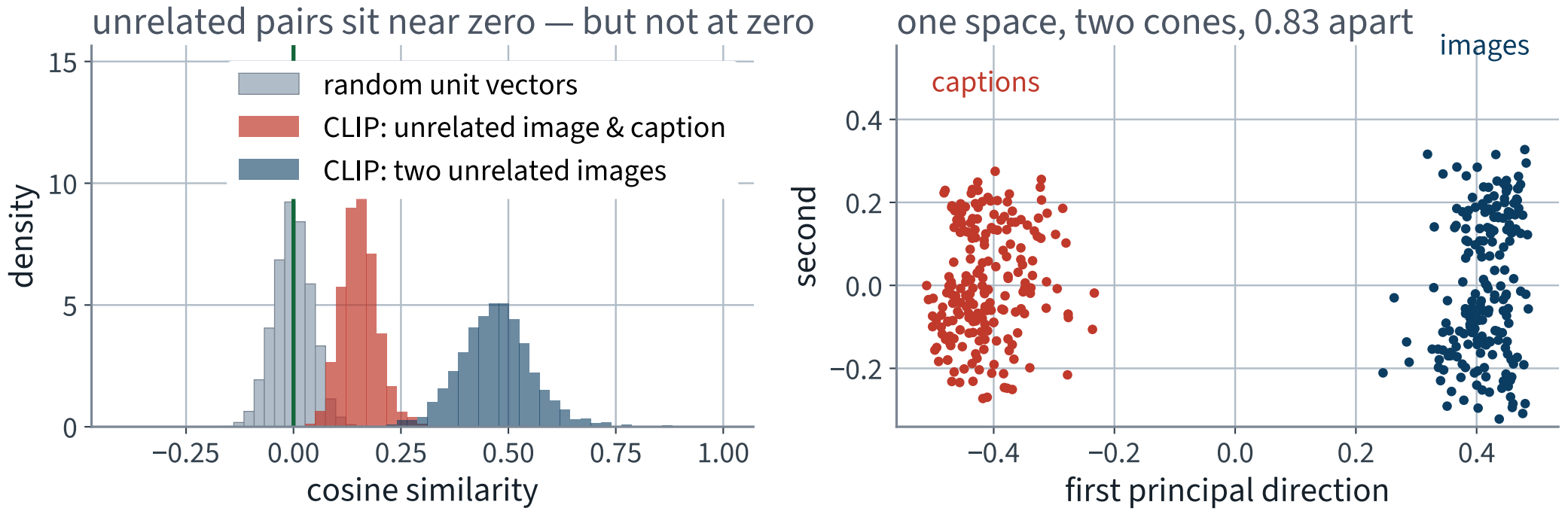
At $d = 512$ the measured spread is 0.0439 against a predicted 0.0442 — and over 20,000 pairs the most negative cosine seen was -0.177.

The number that settles the vote

Random unit vectors in $\mathbb{R}^{512}$, 20,000 pairs	Value
Mean cosine	-0.00072
Standard deviation, measured	0.0439
Standard deviation, $1/\sqrt{d}$	0.0442
Most negative cosine observed	-0.177
Fraction with $\lvert\cos\rvert > 0.5$	0.0%

✗ The most negative cosine in twenty thousand draws was −0.18. Nothing came anywhere near −1, and nothing ever will. That is why the loss targets zero.

One space, two cones



Images occupy one region of the sphere and captions another: the two centroids are 0.83 apart. The objective aligns pairs *relatively*, and never asks the two clouds to coincide.

3 · From a cosine to a logit

Lecture 17 gave us the machinery: softmax over logits, cross-entropy against the correct index. Row i of S is a B-class problem whose correct answer is column i:

$$\mathcal{L} = -\frac{1}{B}\sum_{i} \log \frac{\exp(S_{ii}/\tau)}{\sum_j \exp(S_{ij}/\tau)}$$

INFONCE, ONE DIRECTION; THE LOSS IS THE AVERAGE OF BOTH

Everything is familiar except τ. Set it to 1 and see what happens.

Why $\tau = 1$ cannot work

- The logits are cosines, so they live in $[-1, 1]$. The whole spread of the row is at most 2
- $\exp$ of a range of 2 is a ratio of at most $e^2 \approx 7.4$, across 200 competitors
- So the softmax is nearly uniform whatever the model says, the correct answer gets probability near $1/B$, and the loss sits near $\log B$ — the value for a model that has learned nothing

At $\tau = 1$, on our 200 pairs	Value
Loss	5.151
A scorer that knows nothing: $\log 200$	5.298
Mean probability on the correct entry	0.0058

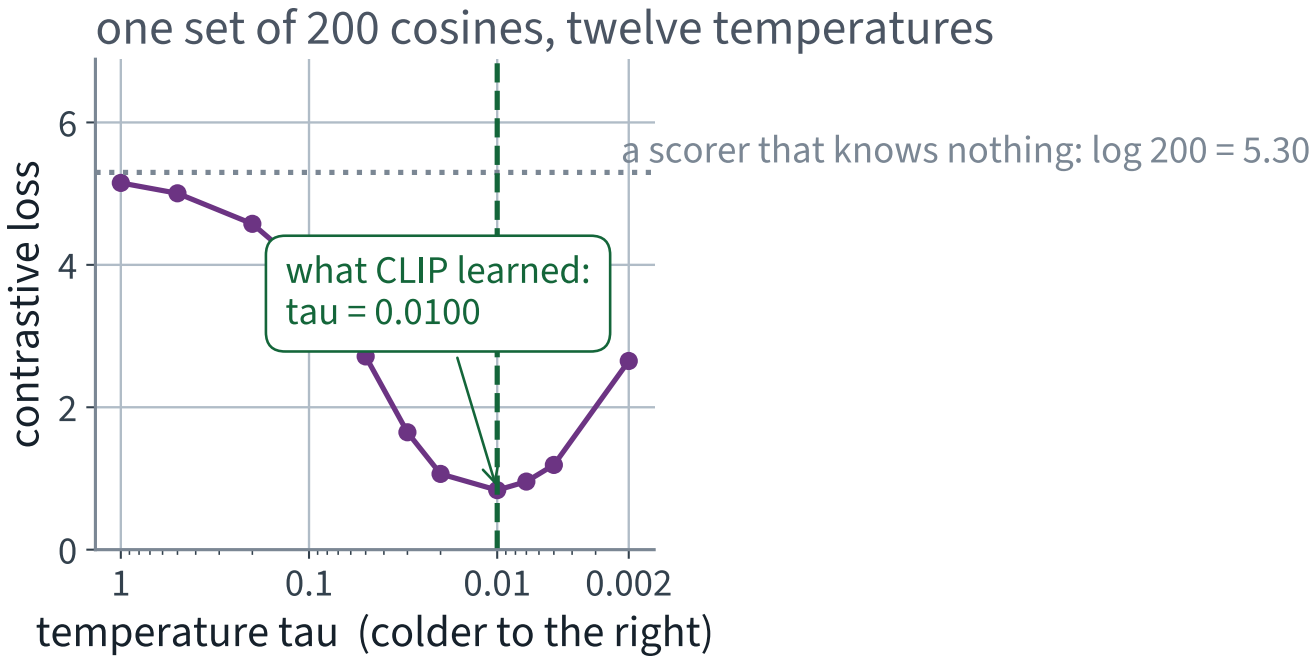
What dividing by τ actually does

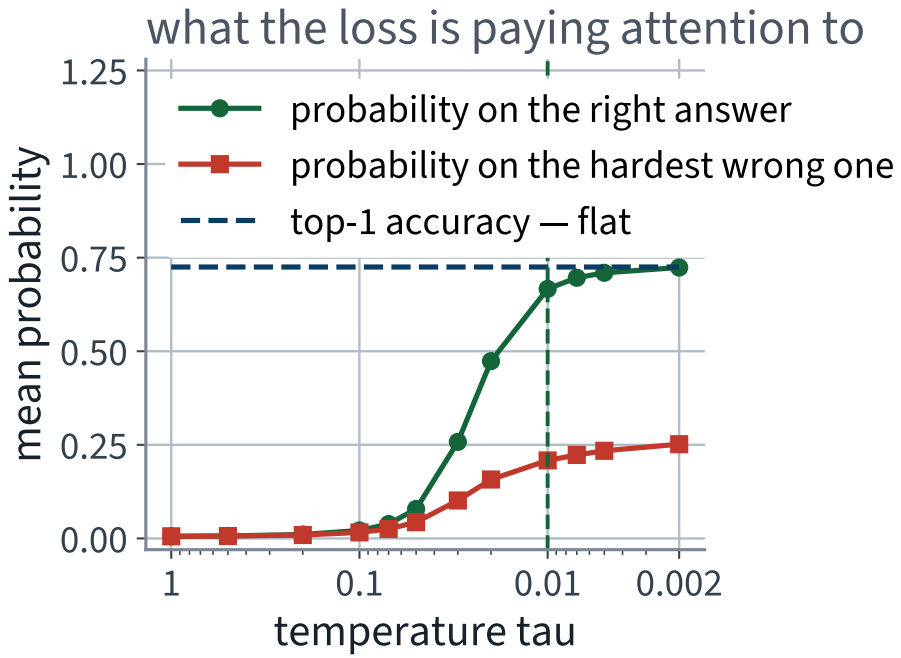
$$\frac{\partial \mathcal{L}_i}{\partial S_{ij}} = \frac{1}{\tau}\left(p_{ij} - \mathbf{1}[j = i]\right), \qquad p_{ij} = \mathrm{softmax}_j(S_{ij}/\tau)$$

LECTURE 17'S RESULT, PREDICTION MINUS TARGET, SCALED BY 1/T

- τ does **not** change which column is largest, so it cannot change the top-1 accuracy of a fixed model
- It changes *where the gradient goes*. Small τ concentrates p on the few hardest negatives; those get almost all the push
- Too small and the loss is dominated by one negative per row, which may be a mislabelled pair rather than an informative one

Twelve temperatures, one set of cosines





The flat line is the point: the temperature moves the loss by orders of magnitude and moves the accuracy of a fixed model not at all.

The temperature is a parameter

It is not a hyperparameter anybody tunes by hand. The model stores $\log(1/\tau)$ and learns it by gradient descent along with everything else, clamped from above so it cannot run away.

```
scale = model.logit_scale.exp()      # a single learnable scalar
print(f"logit scale {scale:.2f}   tau = {1 / scale:.5f}")
```

Read off the checkpoint	Value
Learned logit scale $1/\tau$	100.00
Learned temperature τ	0.01000
Loss on our batch at that τ	0.836
Mean probability on the correct entry	0.667

Read the scale again: 100.00, to five significant figures. It is not near its clamp; it is *on* it. The implementation caps $\log(1/\tau)$ at $\log 100$, and training pushed it there and held it.

And if you make it colder still

τ	Loss	p(correct)	p(hardest wrong)
1	5.151	0.0058	0.0056
✓ 0.0100 (learned)	**✓ 0.836**	**✓ 0.667**	0.208
0.002	2.650	0.724	0.252

Colder is not simply better. As $\tau \to 0$ the softmax becomes a hard maximum: the gradient sees one negative per row, and any label noise in that one pair is amplified rather than averaged away.

4 · Where the negatives come from

Nowhere. That is the elegant part.

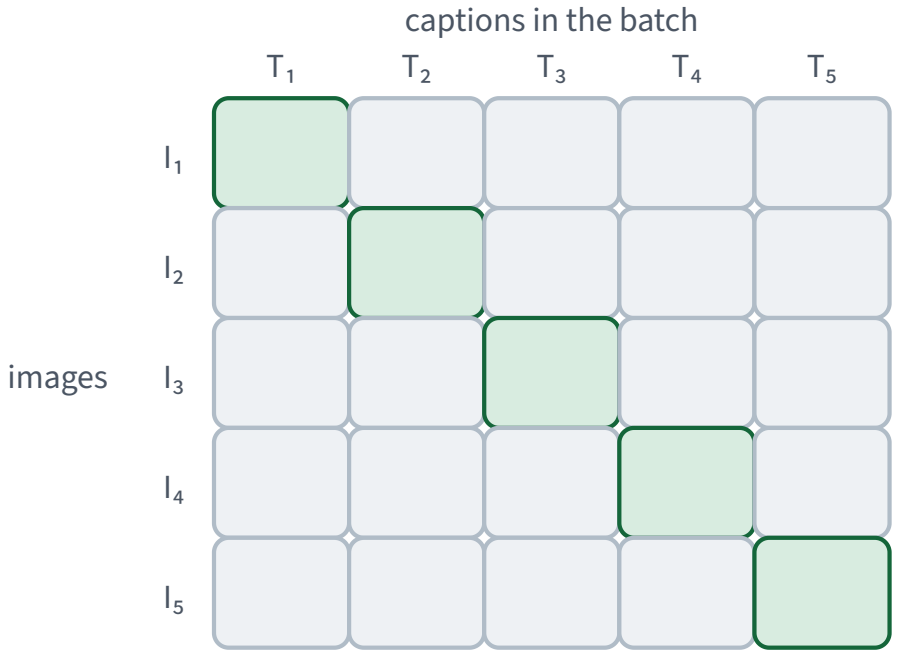
The loss needs, for each image, a set of captions it should *not* match. Nobody labels those. They are simply the other members of the batch — free, and correct with high probability on a large corpus.

> **THE CONSEQUENCE, IN ONE LINE**
>
> The batch is the label set. So the batch size is not a memory setting. It is the number of classes in the classification problem you are solving.

One batch, drawn

One batch of five pairs

every cell is a cosine; the diagonal is the only thing that should win its row



Row 2 is a five-class classification problem

One positive, four negatives, and a softmax across the row — so the batch *is* the label set. Lecture 17 all over again: cross-entropy over logits, where the logits are cosines divided by a temperature.

Bigger batch

more negatives per positive, so a harder task and a lower chance level: 1/B, not 1/5 — the only reason to want one

Not normalised

the cells stop being cosines, the temperature divides a quantity with no fixed scale, and length competes with direction

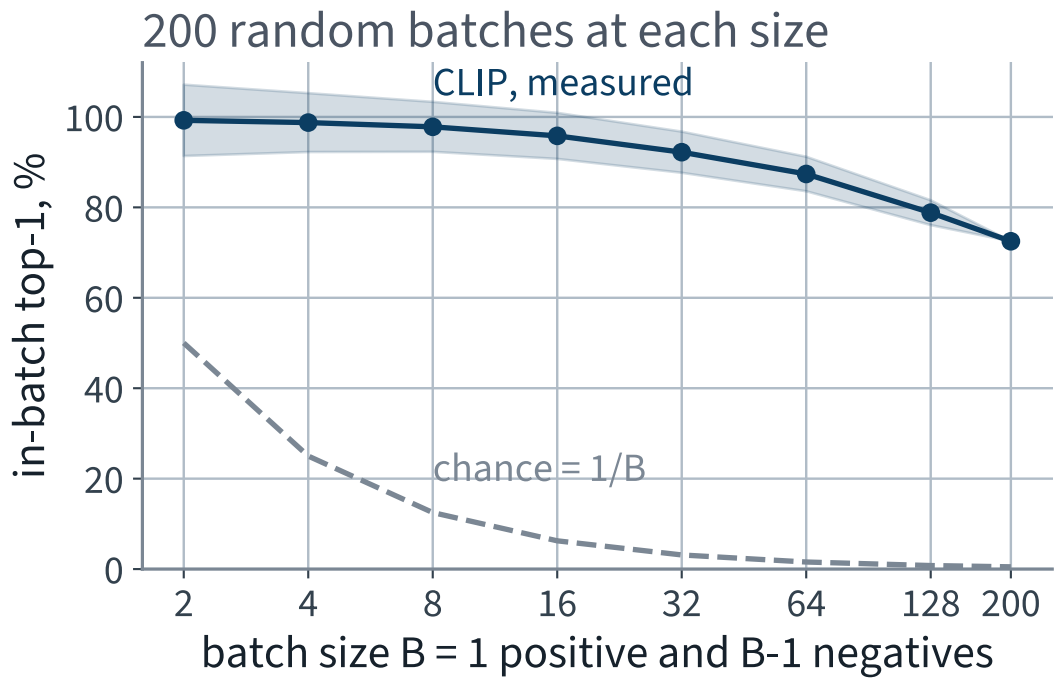
Each row is a B-class problem. Each column is the same problem in the other direction. The loss averages both.

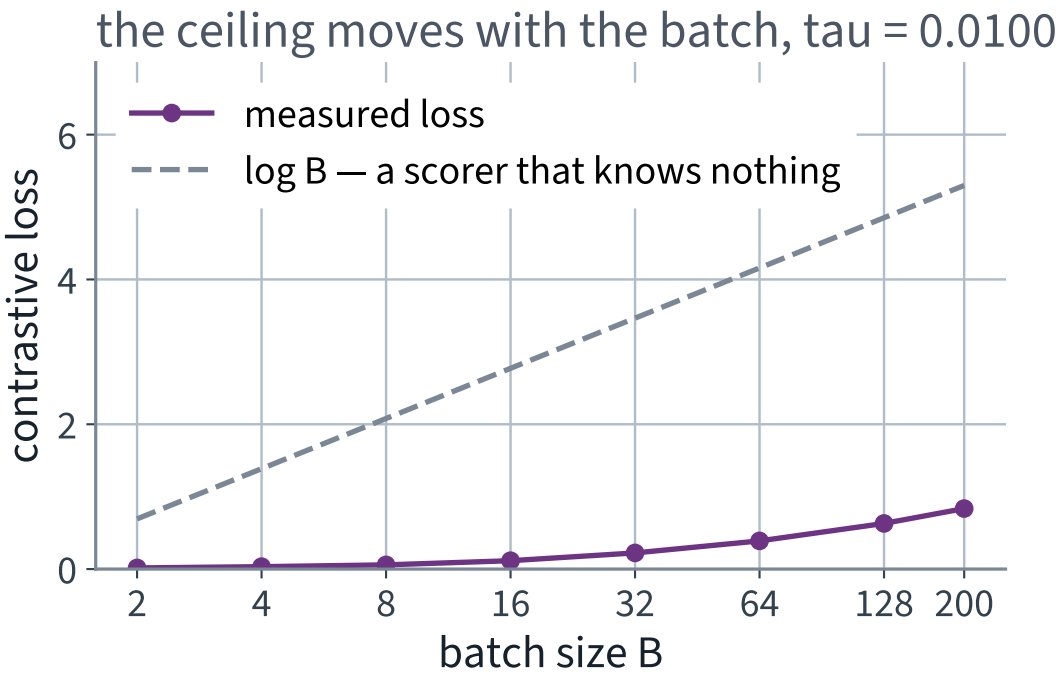
Bigger batch, harder task

Batch size B	In-batch top-1	Chance, $1/B$	Loss
2	99.2%	50.0%	0.016
16	95.8%	6.2%	0.116
64	87.4%	1.6%	0.389
200	72.5%	0.5%	0.836

Same model, same embeddings, same temperature. Only the number of distractors changed.

The number of negatives *is* the difficulty





Accuracy falls with B and chance falls faster, so the gap — the learning signal — widens. That is the whole argument for a large batch.

Two consequences worth carrying away

- **A contrastive loss value is not comparable across papers.** It is measured against a batch-dependent ceiling of $\log B$. Our own sweep makes the point: the same model, the same embeddings and the same τ score 0.058 at B = 8 and 0.836 at B = 200. Neither number means anything without its batch size
- **Batch size stops being an optimisation detail.** Doubling it changes the task, not just the gradient noise — which is why these models are trained with batches in the tens of thousands, across many devices

> **THE ENGINEERING, FLAGGED AS ENGINEERING**
>
> Gathering embeddings across devices to build one large similarity matrix is real and necessary and outside Chapters 1–16. Not examinable.

the derivation — what to keep

1. Normalise, because only the direction is semantic and a bounded score makes one temperature meaningful
2. Unrelated pairs target **zero**, because in d dimensions random directions are orthogonal with spread $1/\sqrt{d}$ — measured at 0.0439 for $d = 512$
3. τ rescales cosines into usable logits. It cannot change a fixed model's ranking; it decides which negatives the gradient listens to
4. The batch *is* the label set, so B sets both the difficulty and the chance level $1/B$

Lecture 8 supplied point 2. Lecture 17 supplied points 3 and 4. The eighteen derivations were meant to be taken in order, and this is why.

THE METHOD

One space, two modalities

25 minutes · examinable

The notebook for this lecture

Open Lecture 23 in Colab

- **Runs on free CPU.** A small pretrained dual encoder, inference only, on a subsampled catalogue
- The two separately-trained encoders measured against each other — the modality gap as a number, not an assertion
- The temperature swept, and its effect on the loss surface plotted

> **WHAT TO CHANGE**
>
> Skip the normalisation before the inner product. Retrieval degrades and the reason is in the derivation: without unit norm the score confounds direction with magnitude, and magnitude carries no meaning here.

The idea, in one paragraph

Take a large collection of (image, caption) pairs. Push each image through an image tower and each caption through a text tower, both ending in a linear map into the *same* d-dimensional space. Normalise onto the unit sphere. Then train so that, within a batch, each image's own caption scores higher than every other caption in the batch.

> **THE DERIVATION, EARLIER TODAY**
>
> Why the sphere; why the unrelated pairs target zero rather than -1; what the temperature does; and why the batch size *is* the difficulty of the task — all four derived in this lecture's mathematics block. Here we use the result and measure it.

The model we will use

Component	What it is
Image tower	ViT-B/32 — the same architecture as before
Text tower	a 12-layer transformer encoder
Shared space	512 dimensions, on the unit sphere
Training data	image–text pairs collected from the web

Non-examinable engineering detail: the checkpoint is about 600 MB and downloads once. The architecture — two encoders and two projections — is entirely Chapters 15–16.

The whole system, in fourteen lines

```python
from transformers import CLIPModel, CLIPProcessor

proc  = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32") \
                 .to(device).eval()

with torch.no_grad():
    ib = proc(images=images, return_tensors="pt").to(device)
    I  = model.get_image_features(**ib).cpu().numpy()
    tb = proc(text=queries, return_tensors="pt",
              padding=True, truncation=True, max_length=77).to(device)
    T  = model.get_text_features(**tb).cpu().numpy()

I, T = unit(I), unit(T)          # onto the sphere, both of them
sim  = T @ I.T                   # (queries, images) — now this is legal
assert sim.shape == (200, 200)
```

Same shape on both sides, so the matrix product exists. That is the entire difference.

One line worth staring at

```
I, T = unit(I), unit(T)
```

`get_image_features` and `get_text_features` return vectors that are *not* normalised. Skip this line and the code still runs, still returns a matrix, and still produces a ranking.

> **A DIFFERENT RANKING**
>
> We measure exactly how different in the next lecture. It is the assistant failure of Lecture 24, and it costs more than you would guess.

Before the catalogue: the known-answer test

The catalogue has no labels, so nothing there can tell us the model is loaded correctly, pointed at the right tensors, and preprocessing the images the way it was trained to.

- Take 2,000 CIFAR-10 test images and the ten class names
- Embed the sentence *“a photo of a {class}”* for each class
- Assign each image to the nearest of the ten sentences

No training, no fitting, no labels used except to score. This is what “zero-shot” means, and it is the check, not the product.

Zero-shot classification, in full

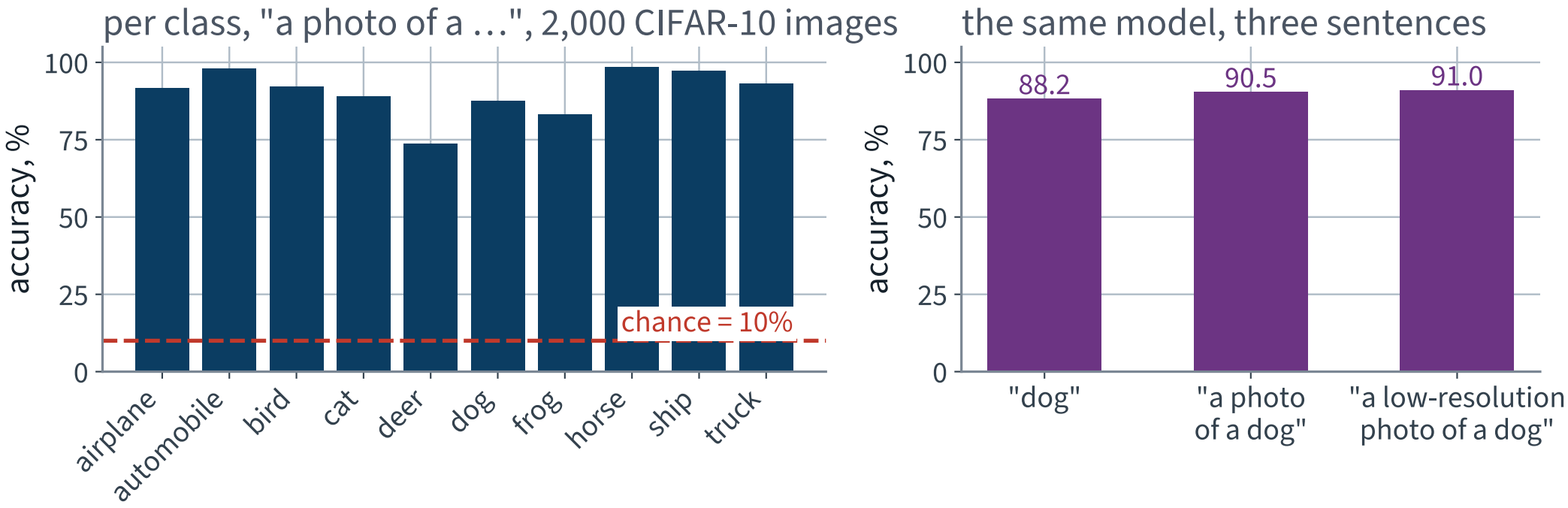
```
classes = ["airplane", "automobile", "bird", "cat", "deer",
           "dog", "frog", "horse", "ship", "truck"]

with torch.no_grad():
    tb = proc(text=[f"a photo of a {c}" for c in classes],
              return_tensors="pt", padding=True).to(device)
    W = unit(model.get_text_features(**tb).cpu().numpy())   # (10, 512)

pred = (F @ W.T).argmax(axis=1)          # F: (2000, 512), already unit
acc  = (pred == y).mean()
print(f"zero-shot accuracy over {len(y)} images: {acc:.1%}   chance: 10.0%")
```

The classifier is ten sentences. There is no fitted parameter anywhere in that cell.

Zero-shot on a set whose labels we have



90.5% over 2,000 images against a 10% chance level — from a model that has never seen CIFAR-10.

The sentence is a hyperparameter

Prompt	Accuracy over 2,000 images
“dog”	88.2%
“a photo of a dog”	90.5%
“a low-resolution photo of a dog”	91.0%
Chance	10.0%

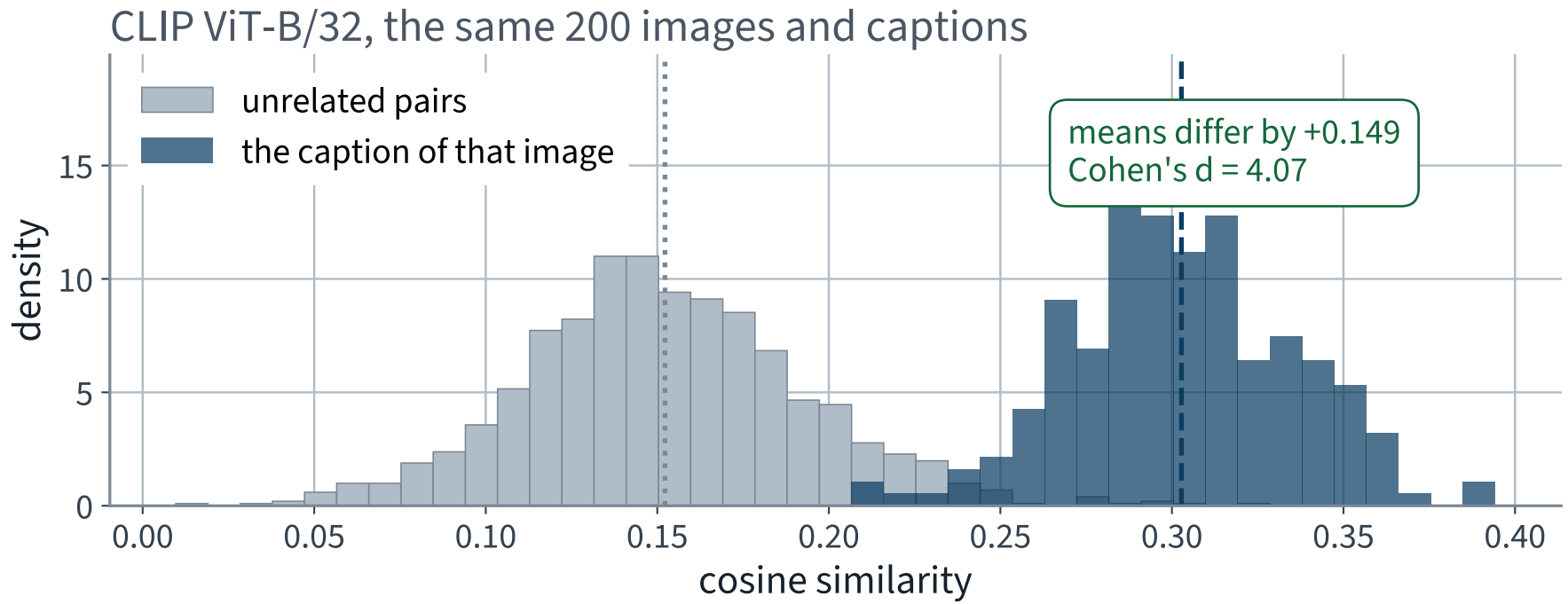
✗ A spread of 2.8% from changing the wording alone. A “zero-shot” number is a number for one particular sentence, and the sentence is a choice you made.

The check passed. Back to the catalogue

Same 200 images. Same 200 queries. Same metric, same ties rule, same baseline. Only the encoders changed.

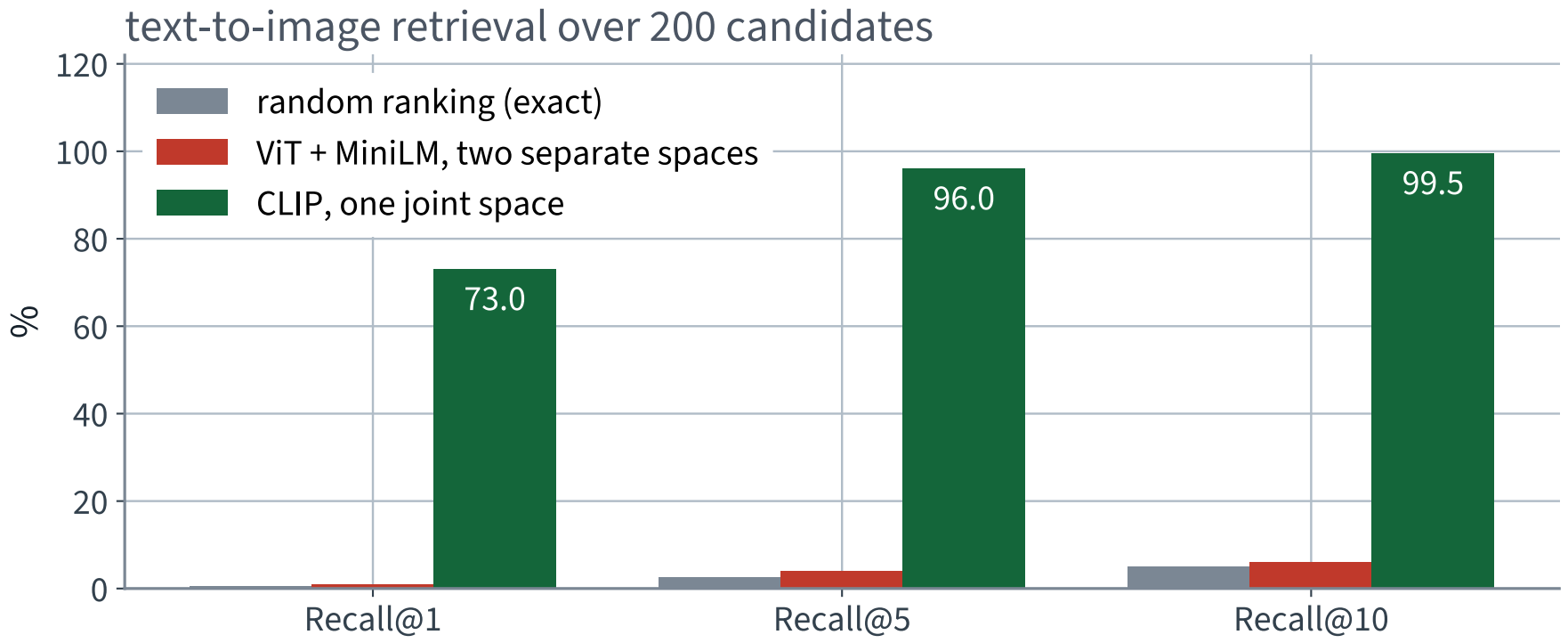
```
rank = (sim >= sim.diagonal()[:, None]).sum(axis=1)   # ties count against us
for k in (1, 5, 10):
    print(f"R@{k:<3d} {(rank <= k).mean():6.1%}   random: {k / len(rank):6.1%}")
print(f"median rank {np.median(rank):.0f} of {len(rank)}")
```

The same measurement, in the joint space



Now the two distributions come apart — the means differ by +0.149, which is 4.1 pooled standard deviations.

Three systems, one metric



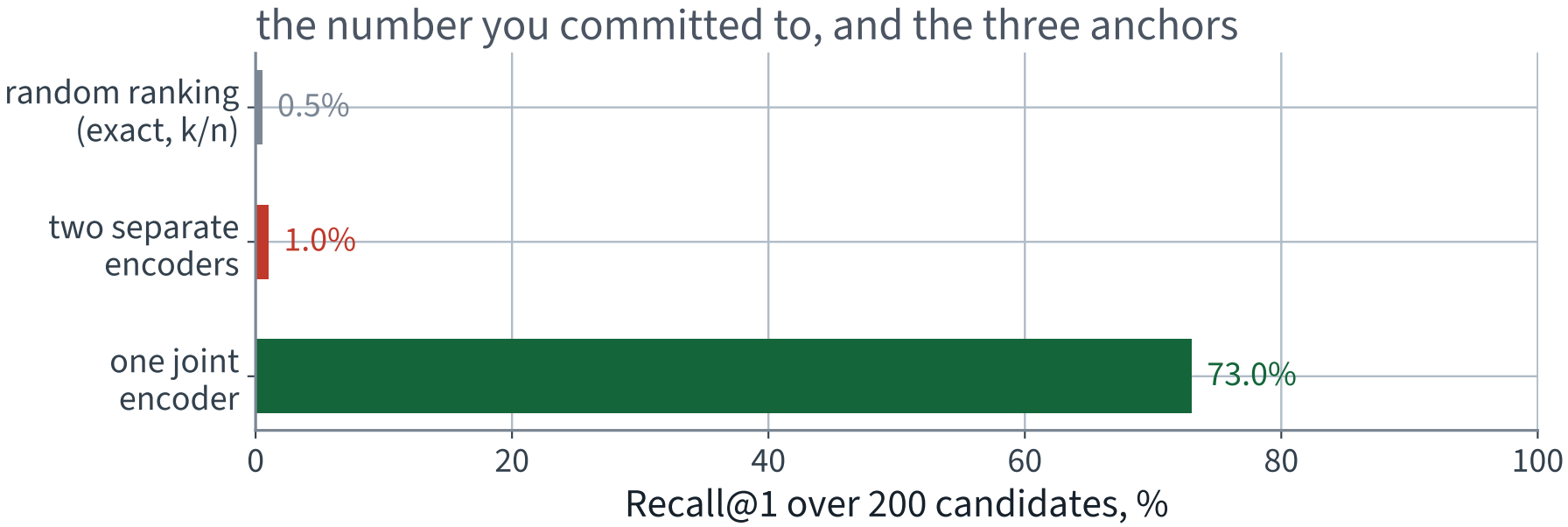
Random ranking and the two-encoder system are the same bar. The joint encoder is a different picture entirely.

The results

System	R@1	R@5	R@10	Median rank	MRR
Random ranking	0.5%	2.5%	5.0%	100	0.029
ViT + MiniLM	**1.0% ✗**	4.0%	6.0%	104	0.037
Joint dual encoder	**73.0% ✓**	**96.0% ✓**	**99.5% ✓**	1	0.827

Text to image, 200 candidates, ties against the model, one caption per entry as the query.

Your committed number, against the anchors



Two of these three bars are the same bar. Which one did your sheet predict?

Rule 4 — report the failure cases

73.0% at rank 1 means **27.0%** of queries did not get the right entry first. Look at them.

- Queries that describe a scene rather than an object — several entries fit equally well, and the “wrong” one is often a reasonable answer
- Fine distinctions the caption makes and the picture barely shows
- Counts and spatial relations: “three”, “behind”, “to the left of”

> **ONE RELEVANT ENTRY IS A FICTION**
>
> Our metric assumes exactly one right answer per query because that is what the data gives us. For “a man riding a horse” on a real catalogue it is simply false, and the measured recall is therefore a *lower* bound.

The queries that have no single answer

Customers do not type captions. They type things like:

- “something to sit on”
- “gear for bad weather”
- “something you would take to the beach”

The right output is not one entry. It is a shortlist with reasons — and we have no way to produce, or to score, a reason.

The second thing the next lecture repairs.

The five questions, one last time

1. **What touched the test set?**
2. **What was fitted, and on what?**
3. **What is the shape here?**
4. **What was dropped — rows, columns, NaNs?**
5. **What is the default I did not ask for?**

Question 3 already earned its place today: the crash on slide 30 *was* question 3, asked by the interpreter instead of by you.

Six ways to leak in a retrieval evaluation

1. The query text is also the indexed description — you are measuring string equality
2. The query's own entry is scored against a shortlist that was built using the query
3. The prompt template was chosen by looking at the number it produced
4. Recall reported without the candidate-set size
5. Ties broken in the model's favour — a constant scorer then reports R@1 = 100%
6. Normalisation applied to one side only, so "cosine" is not a cosine

Audit this

```
sim  = T_emb @ I_emb.T
rank = 1 + (sim > sim.diagonal()[:, None]).sum(axis=1)
print(f"R@1 = {(rank == 1).mean():.1%}")
```

It runs. On the real embeddings it prints a number close to the one on the previous slide. What is wrong with it, and what input would make it print 100%?

Answer

> **STRICT INEQUALITY**
>
> `>` counts only candidates that *beat* the right answer. Anything that ties with it is placed behind it, for free.

Feed it a similarity matrix of all zeros — a model that has learned nothing, or a bug that zeroed the embeddings — and every rank is 1. It reports **R@1 = 100%**.

```
rank = (sim >= sim.diagonal()[:, None]).sum(axis=1)   # ties count against us
```

Audit this too

```python
best = None
for template in ["{}", "a photo of a {}", "a low-resolution photo of a {}"]:
    acc = zero_shot_accuracy(template, X_test, y_test)
    if best is None or acc > best[1]:
        best = (template, acc)

print(f"best template {best[0]!r}: {best[1]:.1%} on the test set")
```

Which of the five questions catches this, and which lecture did we first meet it in?

Answer

QUESTION 1, AND LECTURE 2

Three templates were tried and the best *on the test set* was reported. That is a hyperparameter chosen on the test set — the same error, with a sentence in place of a regularisation strength.

The spread across the three was 2.8%, so the optimism is of that order. The repair is the one you already know: choose the template on a validation split, report the chosen one on the test set, once.

What we did

1. Fed an image to a transformer as a sequence of patches, and named what that gives up: the locality and equivariance convolution assumes
2. Encoded images and text with two separately trained models, and measured that their spaces are *unrelated* — the modality gap
3. Derived why embeddings are normalised onto the sphere, and why unrelated pairs should target zero rather than −1 — the concentration result of Lecture 8, returning
4. Derived the temperature: a cosine lives in $[-1, 1]$ and a softmax over such scores is nearly uniform, so τ rescales them into usable logits
5. Saw that the number of negatives, and so the batch size, sets the difficulty of the task
6. Used a jointly trained dual encoder for zero-shot classification and text-to-image retrieval, evaluated with Lecture 19's metrics

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 23

five, in the style of the written paper

Exercises — Lecture 23

1 Two encoders are trained separately, one on images and one on text. Explain why their cosine similarity carries no information about matching, using a rotation argument. [5]

2 In d dimensions two independent random unit vectors have a cosine with standard deviation $1/\sqrt{d}$. State what an unrelated pair looks like at $d = 512$, and why -1 is the wrong intuition. [5]

3 A contrastive loss is computed at $\tau = 1$ on cosine scores. State why it barely trains, and what τ changes and does not change. [5]

Solutions on Lecture 24's deck.

Exercises — Lecture 23 (continued)

4 Recall@10 is reported over 200 candidates. State the exact random baseline, and why it is computed rather than simulated. [3]

5 An entry with no written description scores zero on a text retrieval route. State whether this is a ranking failure or a structural one, and what follows. [4]

Solutions on Lecture 24’s deck.

THE FIVE SET AT THE END OF LECTURE 22

Solutions Lecture 22

not part of this lecture

Solutions — Lecture 22

1. A recommender is scored by RMSE on held-out ratings. State why that is the wrong metric, using the set the…

✓ RMSE is computed on films the user chose to watch and rate; a recommender operates on the complement.

- It weights every prediction equally when only ten items are shown
- A constant offset ruins RMSE and changes no ranking at all

2. The same model scores HR@10 of 0.0926 and 0.8102 on the same data. Name the two decisions that differ, and…

✓ Random against temporal split, and sampled-100 against the full catalogue. Sampling moves it far more.

- Ten of 101 candidates is already a tenth of the set
- A random ranker goes from 0.0027 to 0.0990 under the same change

Solutions — Lecture 22 (2)

3. Explain why a sampled metric is not merely noisier than a full-catalogue one, and give the consequence for…

✓ The 100 negatives are drawn uniformly from a mostly-tail catalogue, so the distortion depends on the model. Orderings can reverse.

- A popularity model competes against almost no popular items
- A uniform bias would at least preserve rankings; this one does not

4. A sampled softmax draws negatives in proportion to popularity. State the correction, and what enters the…

✓ Subtract $\log q(j)$ from each sampled negative's score. Without it, popularity bias enters the gradient.

- The sample is not the catalogue, and the estimator is biased without reweighting
- The model then systematically under-recommends popular items

Solutions — Lecture 22 (3)

5. A recommender is trained on interactions produced by an earlier recommender. Name the effect, and the earlier…

✓ The feedback loop — the pooling problem of Lecture 19.

- The log records what a previous system chose to show
- A model that would have shown something better scores badly, because the evidence was never collected

LECTURE 24 · GÉRON CH. 15–16

Generation, retrieval-augmented systems, and where this leaves you

Captioning the catalogue entries that have no usable description

Retrieval-augmented generation: Lecture 20’s retriever joined to Lecture 18’s generator, and the failures that belong to the join

The course as one argument, and what it did not cover

The last lecture